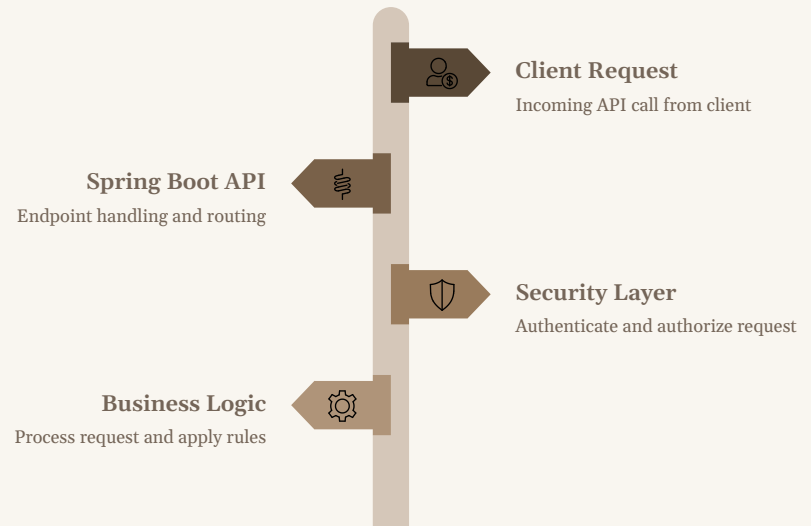




API Security

Authentication + Authorization + Secure Communication

Every API must answer two fundamental questions before serving any request: **Who are you?** and **What are you allowed to do?** In this session, you'll learn how to protect your Spring Boot REST API using industry-standard tools and patterns — without getting lost in deep security theory.



Why API Security?

APIs expose your application's most valuable asset — its data. Without security, anyone on the internet can read, modify, or delete your records. Let's see the difference between an unsecured and a secured API.

✗ Unsecured API

Anyone can call your endpoints with no checks at all.

```
graph TD
    Client --> GET[/students/]
    GET --> Response[200 OK — Anyone can access!]
```

✓ Secured API

Every request passes through authentication and authorization gates.

```
graph TD
    Client --> Auth[Authentication Who are you?]
    Auth --> Authz[Authorization What can you do?]
    Authz --> GET[/students/]
    GET --> Response
```

Authentication

Identifies the caller — proves who is making the request.

Authorization

Controls access — determines what the caller is allowed to do.

HTTPS

Encrypts data in transit so it cannot be intercepted.

Authentication vs. Authorization

These two terms are often confused, but they represent completely different steps in the security process. Think of a hotel: showing your ID at check-in is **authentication**. Your room key only opening your floor is **authorization**.

Authentication — "Who are you?"

Rahul provides his username and password. The system verifies his identity and confirms he is a registered user.

Rahul logs in



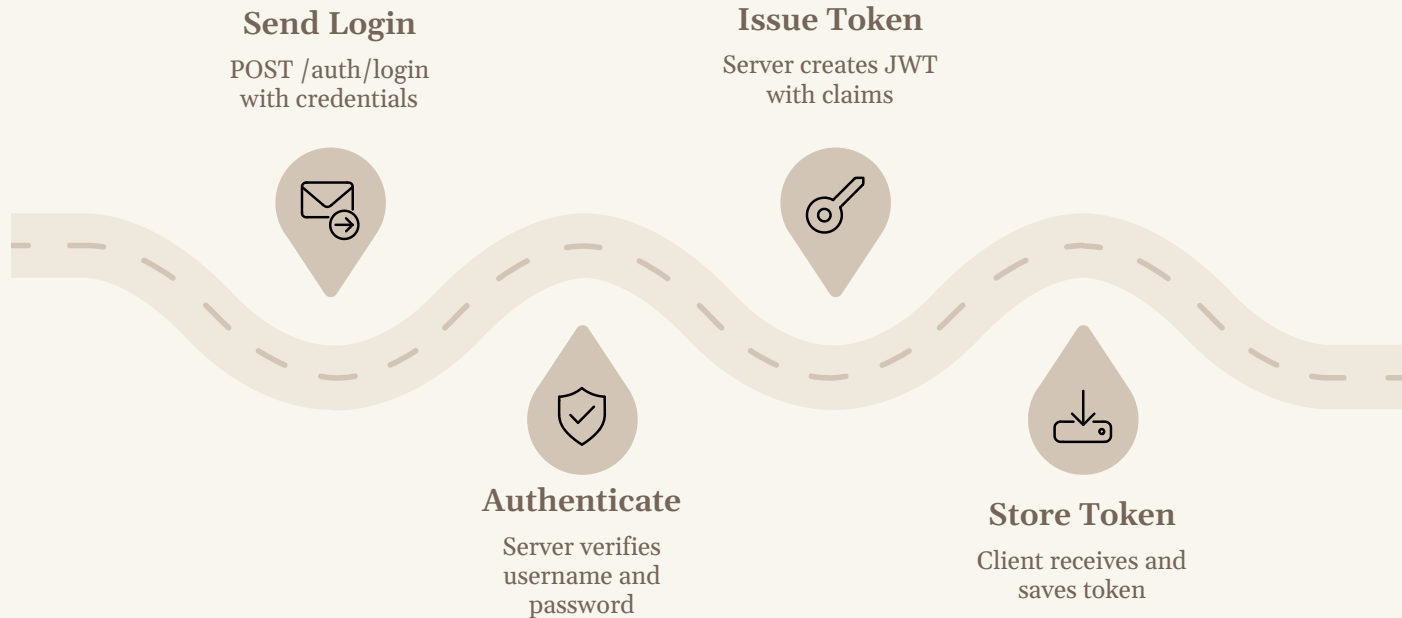
Authentication



Rahul = STUDENT

Login Flow — How JWT Is Issued

The login process is a one-time exchange where the client proves its identity with credentials and receives a token it can use for all future requests. Credentials travel only once — the token travels every time.



Request (Client → Server)

```
POST /auth/login
{
  "username": "rahul",
  "password": "secret"
}
```

Response (Server → Client)

```
200 OK
{
  "token": "eyJhbGciOi..."
}
```

⚠ Never store passwords in plain text — not in the database, not in logs, not anywhere.

What Is JWT?

JWT stands for **JSON Web Token**. It is a compact, self-contained string that carries information about the user — called **claims**. The server generates it on login; the client sends it back on every subsequent request. JWT has three parts separated by dots.

1

Header

Algorithm used to sign the token (e.g. HS256)

```
eyJhbGciOiJIUzI1NiJ9
```

2

Payload

Claims: username, role, user ID, expiry

```
eyJzdWUiOiJyYWh1bCIsc  
InJvbGUiOiJTVFVERU5UI
```

3

Signature

Cryptographic proof the token was not tampered with

```
HMAC_SHA256(  
  header + payload,  
  secretKey  
)
```

✓ Put in JWT

Username, user ID, role, expiry time

✗ Never Put in JWT

Passwords, credit cards, secrets — JWT is **signed, not encrypted** by default

Using JWT — The Complete Request Flow

After login, the client stores the token and attaches it to every subsequent API call using the `Authorization` header. Spring Security intercepts the request, validates the token, and either allows or rejects the call — before your controller ever runs.

Step 1 — Login Once

```
POST /auth/login
username + password
↓
Authentication
↓
JWT issued
↓
Client stores token
```

Step 2 — Use Token Every Time

```
GET /students
Authorization: Bearer eyJhbGci...
↓
Spring Security
↓
Validate JWT
↓
Allow
```



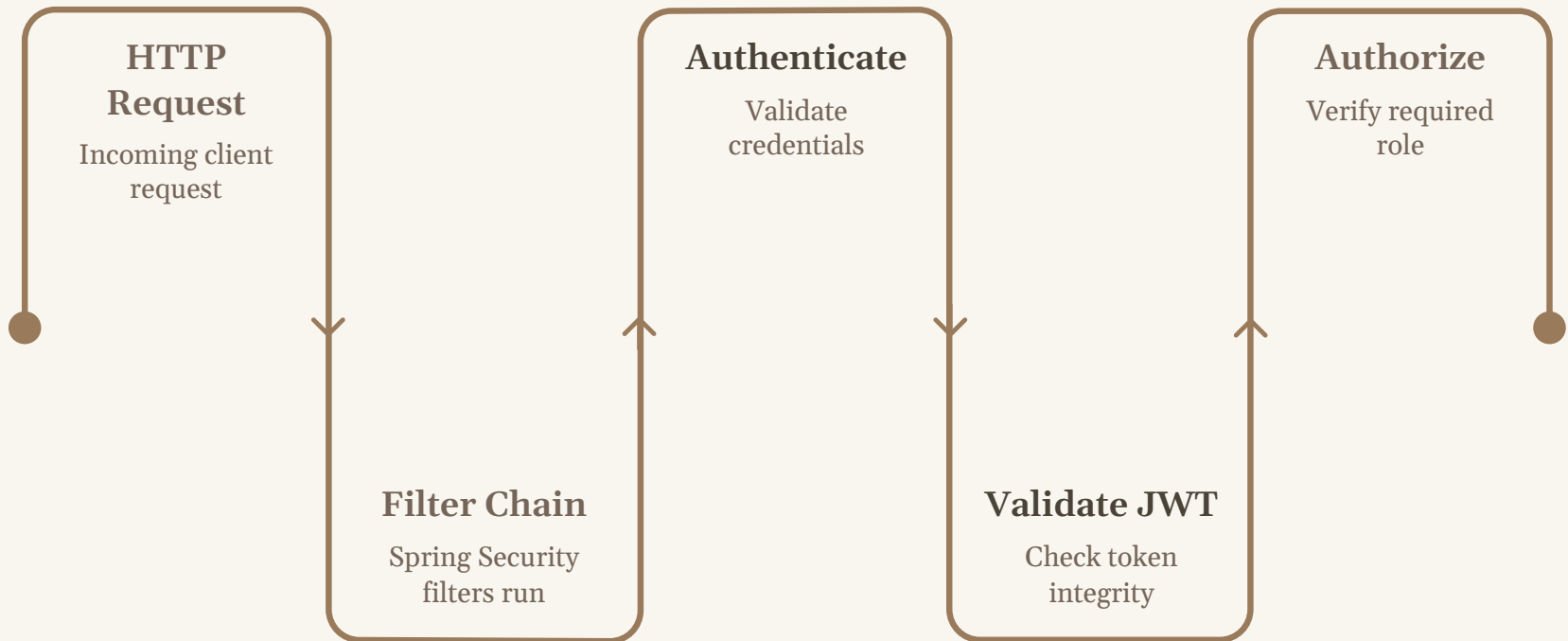
Role-Based Authorization

Once a user is authenticated, Spring Security checks their **role** before allowing access to specific endpoints. Different roles get different permissions. This is called Role-Based Access Control (RBAC).

ADMIN Role

Spring Security Architecture

Spring Security works as a **filter chain** that wraps your application. Every HTTP request must pass through this chain before it ever reaches your controller. You don't need to understand every filter — just know that security runs first, always.



Key Insight

Spring Security intercepts requests **before** your controllers execute. If authentication or authorization fails, the request never reaches your business logic at all.

What You Configure

You define which endpoints are public (e.g. `/auth/login`) and which are protected, and which roles can access each route — Spring handles the rest automatically.

Password Security — Never Store Passwords Directly

Storing passwords as plain text is one of the most dangerous mistakes a developer can make. If your database is ever leaked, every user's password is exposed immediately. The solution is **BCrypt hashing**.

❌ Bad — Plain Text

Database row:
username = rahul
password = secret123

If DB is leaked →
ALL passwords exposed instantly!

✅ Good — BCrypt Hash

Database row:
username = rahul
password = \$2a\$10\$N9qo8uLOickgx2ZMRZoMye...

If DB is leaked →
Hash cannot be reversed



❏ BCrypt is intentionally slow to compute — this makes automated password-guessing (brute force) attacks significantly harder and more expensive.

CORS, CSRF & HTTPS

Three more security concepts every Spring Boot developer should know at a basic level. You don't need to master them today — but you do need to know what they are and why they matter.



CORS — Cross-Origin Resource Sharing

Controls which browser origins are allowed to call your API. Your React frontend on `localhost:3000` talking to Spring Boot on `localhost:8080` is a cross-origin request — CORS rules must explicitly allow it.



CSRF — Cross-Site Request Forgery

Mainly relevant for browser apps using cookie-based sessions. For stateless JWT APIs using `Authorization` headers, CSRF protection is typically disabled intentionally — but only when the architecture makes that safe.



HTTPS / TLS

HTTP sends data in plain text — anyone intercepting the connection can read it. HTTPS encrypts all communication between client and server. **Always use HTTPS in production.** Never send credentials over HTTP.

Security Rules Every Developer Must Follow

These are non-negotiable best practices. Violating even one of them can expose your application and your users to serious risk. Memorize these and apply them on every project.



Hash passwords with BCrypt



Always use HTTPS in production



Validate JWT on every request



Check authorization roles before serving data



Configure CORS carefully — don't allow all origins



Never commit passwords or API keys to Git



Never put secrets in `application.properties` in Git



Do not put passwords or sensitive data inside JWT



Return 401 for authentication failure



Return 403 for authorization failure

Hands-On: Secure the Student API

Now it's your turn. Over the next 35–40 minutes, you'll take the Student Management REST API you already built and make it production-grade secure. Follow each step carefully and test as you go.

01

Add Spring Security dependency

Add `spring-boot-starter-security` and JWT libraries to `pom.xml`

02

Create Users with roles

Define ADMIN and STUDENT users in the database with assigned roles

03

Hash passwords with BCrypt

Use `BCryptPasswordEncoder` — never store raw passwords

04

Configure Authentication

Set up `AuthenticationManager` and login endpoint `POST /auth/login`

05

Add JWT generation & validation

Create `JwtUtil` class to issue and validate tokens

06

Protect REST endpoints by role

Use `@PreAuthorize` or `SecurityFilterChain` to restrict routes

07

Test 401 / 403 / 200 responses

Use Postman to verify each role gets the correct HTTP response

Target Architecture

```
graph TD
    Client --> JWTToken[JWT Token]
    JWTToken --> SpringSecurity[Spring Security]
    SpringSecurity --> ValidateToken[Validate Token]
    ValidateToken --> RoleAuthorization[Role Authorization]
    RoleAuthorization --> CheckPermissions[Check Permissions]
    CheckPermissions --> StudentController[Student Controller]
    StudentController --> ServiceLayer[Service Layer]
    ServiceLayer --> Database[Database]
```

Expected Test Results

GET /students (STUDENT) → 200